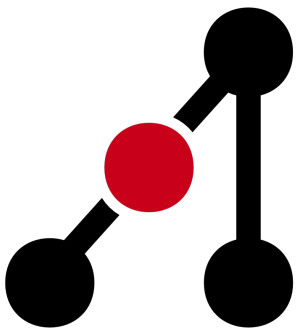
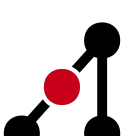


Adelaide Competitive Programming Club

Data Structures Workshop

Data Structures for Competitive Programming

Stack



Stack

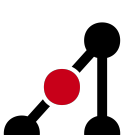
A stack is **Last In, First Out (LIFO)** — think a stack of plates. You can only add or remove from the top, you can't pull one out from the middle!

The two core operations are `push` (add to top) and `pop` (remove from top). You'll also often want to **peek** at the top without removing it.

In Python, a regular `list` works perfectly as a stack:

```python
stack = []

stack.append(1)   # push 1  -> [1]
stack.append(2)   # push 2  -> [1, 2]
stack.append(3)   # push 3  -> [1, 2, 3]

top = stack[-1]   # peek    ->  3  (no removal)
val = stack.pop() # pop     ->  3, stack = [1, 2]
```

```
if not stack:     # empty check
    print("Stack is empty")
```

Stack — When to use it & Complexity

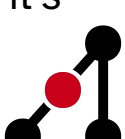
Stacks show up a lot in competitive programming! Some classic use cases:

- Matching parentheses / brackets
- Undo / redo operations
- Monotonic stack problems

Everything is nice and fast:

Operation	Time
`push` (append)	`O(1)*`
`pop`	`O(1)`
peek `[-1]`	`O(1)`
search	`O(n)`

The `*` on O(1) means it's **amortised** — occasionally Python has to resize the list under the hood, but it's O(1) on average.



Stack — Practice

Easy

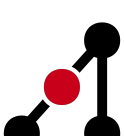
- 20. Valid Parentheses
- 682. Baseball Game
- 1544. Make The String Great

Medium

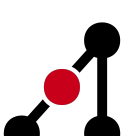
- 739. Daily Temperatures
- 150. Evaluate Reverse Polish Notation
- 735. Asteroid Collision

AUCPL

- Tung Tung Sahur (Div B)
- End of the Line (Div A)



Queue



Queue

A queue is **First In, First Out (FIFO)** — think a line at a shop. The first person to join is the first to be served.

You add to the back (`enqueue`) and remove from the front (`dequeue`). This is the opposite ends to a stack!

You might think to use a list here too, but `list.pop(0)` is O(n) because it has to shift every element. Instead, use `collections.deque` which is O(1) from both ends:

Python

```python
from collections import deque

q = deque()
q.append('A')        # enqueue - add to back  - O(1)
q.append('B')
first = q.popleft()  # dequeue - remove front - O(1)
```

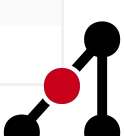
You will use this constantly for BFS!

Queue — BFS Skeleton

The most common use of a queue in competitive programming is BFS (Breadth-First Search). Here's a template you can basically copy-paste:

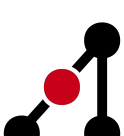
Python

```python
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue   = deque([start])

    while queue:
        node = queue.popleft()   # grab the next node to process
        print(node)

        for neighbour in graph[node]:
            if neighbour not in visited:
```

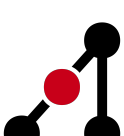



```
                visited.add(neighbour)
                queue.append(neighbour)  # schedule it to be visited
```

The `visited` set stops us from going in circles. We'll see this pattern come up a lot!



Deque

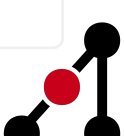


Deque

A **Double-Ended Queue** (deque, pronounced “deck”) gives you O(1) insert and remove from **both** ends. It’s like a stack and a queue had a child.

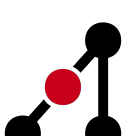
You’ve already seen `deque` used as a queue above — but it can do more:

```python
from collections import deque

d = deque([1, 2, 3])

d.appendleft(0)  # [0, 1, 2, 3]    - add to front
d.append(4)      # [0, 1, 2, 3, 4] - add to back

d.popleft()      # removes 0  -> [1, 2, 3, 4]
d.pop()          # removes 4  -> [1, 2, 3]
```



```
d.rotate(1)      # [3, 1, 2]  (rotate right by 1)
d.rotate(-1)     # [1, 2, 3]  (rotate left by 1)

d[0]             # peek front - O(1)
d[-1]            # peek back  - O(1)
```

The main use case is sliding window problems, where you need to efficiently add to one end and remove from the other.



Queue & Deque — Practice

Easy

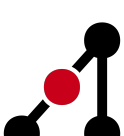
- 933. Number of Recent Calls
- 2073. Time Needed to Buy Tickets

Medium

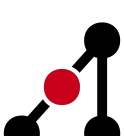
- 239. Sliding Window Maximum
- 102. Binary Tree Level Order Traversal
- 994. Rotting Oranges

AUCPL

- Feeding Frenzy (Div A)
- Merch Shipment (Div B)



Set



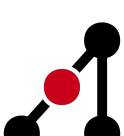
Unordered Set (`set`)

A set stores a collection of **unique** values. It's your go-to whenever you need to quickly answer "have I seen this before?"

All the core operations are O(1) on average — it does **not** keep things in any particular order though.

```python
s = set()
s.add(3)        # O(1) avg
s.remove(3)     # O(1) avg - raises KeyError if 3 isn't there
s.discard(99)   # O(1)     - safe version, no error if missing

3 in s          # O(1) membership check - this is the killer feature!
```

Remember the `visited` set in our BFS template? That's a perfect example of a set in action.



Ordered Set (`SortedList`)

Sometimes you need a set that **also** keeps its elements sorted. Python doesn't have one built in, but `sortedcontainers` (available on most contest judges) gives us `SortedList`:

```python
from sortedcontainers import SortedList

sl = SortedList()
sl.add(5)      # O(log n)
sl.add(2)
sl.add(8)
# sl = [2, 5, 8]  - always sorted, no matter what order you insert!

sl.bisect_left(5)   # what index would 5 go at? -> 1
sl[0]               # min element -> 2
sl[-1]              # max element -> 8
sl.remove(5)        # O(log n)
```

The tradeoff vs a regular set is that operations are O(log n) instead of O(1), but you get the sorted order for free. Use this when you need to find the floor, ceiling, or rank of elements.

Set — Practice

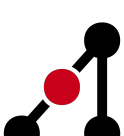
Easy

- 217. Contains Duplicate
- 202. Happy Number
- 349. Intersection of Two Arrays

Medium

- 128. Longest Consecutive Sequence
- 454. 4Sum II

heapq & bisect



heapq — Priority Queue

Sometimes you don't just want the next element in line — you want the **most important** one. That's what a priority queue gives you, and Python implements it as a **min-heap** via `heapq`.

The smallest element is always at index `0`, and you can push/pop in O(log n):

```python
import heapq

h = []
heapq.heappush(h, 5)
heapq.heappush(h, 2)
heapq.heappush(h, 8)

heapq.heappop(h)   # -> 2 (always the smallest)
h[0]               # peek at smallest without removing - O(1)
```

```
# Already have a list? Convert it to a heap in O(n):
nums = [5, 2, 8, 1]
heapq.heapify(nums)
```


heapq — Priority Queue

Python only has a min-heap, but to mimic a max-heap you can negate your values

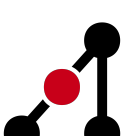
```python
# Simulate a max-heap by negating everything
heapq.heappush(h, -val)
max_val = -heapq.heappop(h)
```

You can also push tuples — Python compares them element by element, so the first value acts as the priority:

```python
heapq.heappush(h, (priority, item))
```

When to use:

- K-th largest / smallest element
- Dijkstra's shortest path algorithm
- Any greedy problem where you always process the min or max next



bisect — Free Binary Search

Writing binary search from scratch is painful and easy to get wrong. The `bisect` module does it for you on any sorted list in O(log n).

Python

```python
import bisect

a = [1, 3, 5, 7, 9]

# Where would 5 go to keep the list sorted?
bisect.bisect_left(a, 5)   # -> 2  (before existing 5s)
bisect.bisect_right(a, 5)  # -> 3  (after existing 5s)
# Actually insert a value and keep sorted - O(n) due to the shift
bisect.insort(a, 4)        # a = [1, 3, 4, 5, 7, 9]
# Count how many times val appears in a sorted list
def count_val(a, val):
    return bisect.bisect_right(a, val) - bisect.bisect_left(a, val)
```

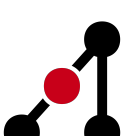
bisect — Free Binary Search

Writing binary search from scratch is painful and easy to get wrong. The `bisect` module does it for you on any sorted list in O(log n).

The `left` vs `right` distinction matters when there are duplicates, but the main thing to memorise is what each gives you:

What you want	How to get it
First index ≥ x	`bisect_left(a, x)`
First index > x	`bisect_right(a, x)`
Count of elements < x	`bisect_left(a, x)`
Count of elements ≤ x	`bisect_right(a, x)`
Count of elements = x	`bisect_right(a,x) - bisect_left(a,x)`

Once you get the hang of it, `bisect` saves a lot of time. It's a great thing to put in your notes!



heapq — Practice

Easy

- 1046. Last Stone Weight
- 703. Kth Largest Element in a Stream

Medium

- 215. Kth Largest Element in an Array
- 621. Task Scheduler
- 973. K Closest Points to Origin

AUCPL

- Pumpkinomics

bisect — Practice

Easy

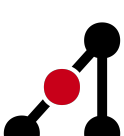
- 35. Search Insert Position
- 1351. Count Negative Numbers in a Sorted Matrix

Medium

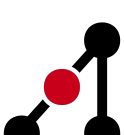
- 34. Find First and Last Position
- 981. Time Based Key-Value Store
- 1011. Capacity To Ship Packages

AUCPL

- Sausge Slinger



Map



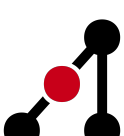
Unordered Map (`dict`)

You've already seen Python dicts, but they're worth covering properly. A dict maps **keys** to **values** and lets you look them up in O(1). They're incredibly useful for counting, memoisation, and grouping.

Python

```python
from collections import defaultdict, Counter

d = {}
d["key"] = 42      # insert
d["key"]           # lookup - crashes with KeyError if key is missing!
d.get("x", 0)      # safe lookup - returns 0 if "x" isn't there

# defaultdict saves you from KeyError when counting things
freq = defaultdict(int)
for ch in "hello":
    freq[ch] += 1   # no need to check if ch is already a key
```

```python
# Counter is even more convenient for frequency counting
cnt = Counter("hello")
# Counter({'l': 2, 'h': 1, 'e': 1, 'o': 1})
cnt.most_common(2)  # top-2 most frequent: [('l', 2), ('h', 1)]
```

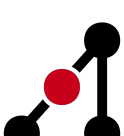
Ordered Map (`SortedDict`)

Just like `SortedList`, sometimes you want a dict that also keeps its **keys** sorted. `SortedDict` from `sortedcontainers` does exactly this:

```python
from sortedcontainers import SortedDict

sd = SortedDict()
sd[3] = "three"
sd[1] = "one"
sd[5] = "five"
# No matter what order you insert, the keys stay sorted: [1, 3, 5]

sd.peekitem(0)    # smallest key: (1, "one")
sd.peekitem(-1)   # largest key:  (5, "five")

# Range query: all entries with keys in [2, 4]
```

```
idx_l = sd.bisect_left(2)
idx_r = sd.bisect_right(4)
sd.keys()[idx_l:idx_r]  # [3]
```

This is useful when you need to answer questions like “what’s the nearest key to X?” or “give me all keys between A and B”.



Map (dict) — Practice

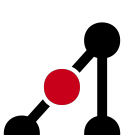
Easy

- 1. Two Sum
- 383. Ransom Note
- 169. Majority Element

Medium

- 49. Group Anagrams
- 560. Subarray Sum Equals K
- 438. Find All Anagrams in a String

Quick Reference



Quick Reference

Here's a cheat sheet of everything we covered today. Print this out and stick it in your notes!

Structure	Python Type	Key Operations	Time	Best For
Stack	`list`	`append` / `pop`	`O(1)*`	DFS, matching parens, monotonic
Queue	`collections.deque`	`append` / `popleft`	`O(1)`	BFS, level-order, scheduling
Deque	`collections.deque`	`append/left` , `pop/left`	`O(1)`	Sliding window, palindrome
Set	`set`	`add` / `remove` / `in`	`O(1)*`	Visited, membership, de-dup
Ordered Set	`SortedList`	`add` / `remove` / `[]`	`O(log n)`	Floor/ceil, range count
heapq	`heapq` + `list`	`heappush` / `heappop`	`O(log n)`	Dijkstra, K-th, greedy
bisect	`bisect` + sorted list	`bisect_left` / `right`	`O(log n)`	Lower/upper bound, count ≤ x
Dict	`dict` / `Counter` / `defaultdict`	`d[k]` / `get` / `in`	`O(1)*`	Frequency, lookup, memoize
Ordered Map	`SortedDict`	`d[k]` / `peekitem` / `bisect`	`O(log n)`	Floor/ceil queries, range keys